TEMA 1 – INTRODUCCIÓN AL MODELO DE OBJETOS DOM

DOM

Al presentar un documento, el navegador va leyendo el código HTML línea por línea y aprovecha para **construir una representación jerárquica suya estructurada en nodos**, que se denomina DOM (*Document Object Model*).

En cierta forma podríamos pensar en el DOM de un documento como si fuera el árbol genealógico de una familia, en el que:

- **el nodo raíz es el documento completo**,
- de él depende el nodo html,
- de él los nodos head y body,
- y así sucesivamente.

¿Por qué se toma esta molestia el navegador?

Porque **pone este DOM a disposición de JavaScript para que pueda interactuar con él** como si se tratara de un objeto más. JavaScript no posee por sí mismo la capacidad de interactuar con la página, sino que la obtiene a través del DOM.

Los nodos del DOM pueden ser de diversos tipos, pero los más habituales son:

- Documento. Es el nodo raíz de toda la página.
- Tipo de documento. En el caso de las páginas web, este elemento corresponde a la directiva `<!DOCTYPE html>`.
- Elemento. Cada elemento HTML se convierte en un nodo de este tipo.
- Texto. Los textos de los elementos HTML se convierten en nodos de este tipo.
- Atributo. Cada atributo de un elemento HTML se convierte en un nodo de este tipo. Obviamente, los nodos de este tipo serán hijos de nodos de elemento.
- Esos tipos de nodos se identifican mediante constantes, que se recogen en la tabla que sigue:

Constante (Tipo de nodo)	Significado (hipervínculo al foro W3C)	Valor
ELEMENT_NODE	El nodo es del tipo Element	1
ATTRIBUTE_NODE (deprecated)	El nodo es del tipo Attr	2
TEXT_NODE	El nodo es del tipo Text	3
CDATA_SECTION_NODE (deprecated)	El nodo es del tipo CDATASection	4
ENTITY_REFERENCE_NODE (deprecated)	El nodo es del tipo EntityReference	5
ENTITY_NODE (deprecated)	El nodo es del tipo Entity	6
PROCESSING_INSTRUCTION_NODE	El nodo es del tipo ProcessingInstruction	7
COMMENT_NODE	El nodo es del tipo Comment	8
DOCUMENT_NODE	El nodo es del tipo Document	9
DOCUMENT_TYPE_NODE	El nodo es del tipo DocumentType	10
DOCUMENT_FRAGMENT_NODE	El nodo es del tipo DocumentFragment	11

Fuente: W3C

Y ¿cómo puede acceder JavaScript a estos nodos del DOM?

Existen diversas técnicas que **varían en función del tipo de nodo** y que trataremos con detalle en el tema siguiente, pero nos interesa adelantar aquí que:

- el navegador crea automáticamente un objeto JavaScript llamado `document`,
- que el objeto `document` posee un método llamado `getElementById(`*`id_del_`*`nodo)` que devuelve un objeto asociado al nodo de tipo elemento cuyo valor para el atributo `id` coincide con *`id_del_nodo`*.

Por ejemplo, `document.getElementById('titulo')` devolvería un objeto asociado al elemento HTML cuyo `id` es `titulo`.

¿Y qué podemos hacer con estos objetos?

Una vez más **dependerá del tipo de nodo**, pero **todos ellos comparten**:

- algunas propiedades comunes
- Algunos métodos comunes

pues todos ellos **heredan de un mismo objeto** llamado **`Node`**.

En la siguiente tabla se describen algunas propiedades de interés que nos ayudarán a entender el ejemplo de código que se presenta a continuación.

Propiedad	**Descripción**
nodeName	Indica el nombre del nodo. En el caso de un nodo de elemento su nombre es su etiqueta HTML, mientras que en el caso de un nodo de texto su nombre es la cadena '#text'.
nodeType	Indica el tipo de nodo con un número (referenciado en la tabla anterior): 9 → Nodo de tipo documento 10 → Nodo de tipo "tipo de documento" 1 → Nodo de tipo elemento 2 → Nodo de tipo atributo 3 → Nodo de tipo texto
childNodes	Devuelve una **colección** de todos los nodos hijos del nodo, **exceptuando los de tipo atributo**, para los que existe otra forma diferente de acceso. (una **colección** es una entidad similar a un **array** y que, en esencia, se maneja del mismo modo)
nodeValue	Devuelve o establece el valor de los nodos: ➢ En un **nodo de tipo documento** el nodeValue es null, ➢ en uno de **tipo texto** el nodeValue es el propio texto, y ➢ en uno de **tipo atributo** el nodeValue es el valor del atributo.

Adicionalmente,

- todos los nodos de tipo elemento heredan las propiedades y métodos del objeto `HTMLElement`, y
- cada tipo de elemento hereda también de un objeto concreto;

por ejemplo:

el elemento `textarea` hereda del objeto DOM `HTMLTextAreaElement` que, entre otras, posee una propiedad llamada `value` que nos permite acceder y establecer el contenido del área de texto.

Syntax

value = node.nodeValue;

value is a string containing the value of the current node, if any.

Notes

For the document itself, nodeValue returns null.

For text, comment, and CDATA nodes, nodeValue returns the content of the node.

For attribute nodes, the value of the attribute is returned.

The following table shows the return values for different elements:

Attr	**value of attribute**
CDATASection	content of the CDATA Section
Comment	content of the comment
Document	null
DocumentFragment	null
DocumentType	null
Element	null
NamedNodeMap	null
EntityReference	null
Notation	null
ProcessingInstruction	entire content excluding the target
Text	content of the text node

When nodeValue is defined to be null, setting it has no effect.

Por ejemplo, la siguiente página web está compuesta por un botón y un área de texto. Al pulsar el botón se ejecuta un código JavaScript que recorre el árbol DOM del documento y los presenta dentro del área de texto.

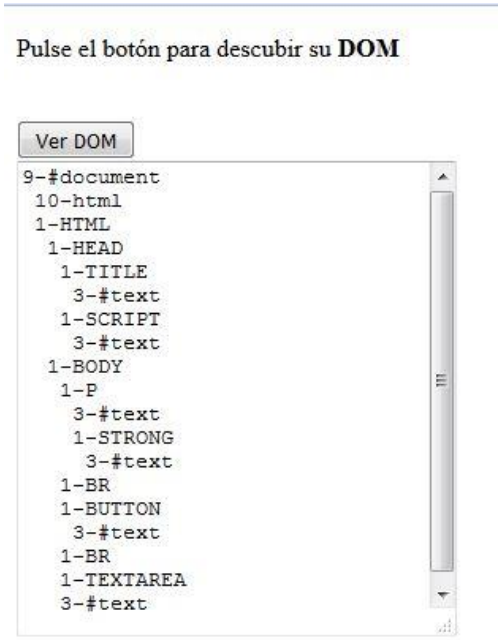
```
001    <!DOCTYPE html>
002    <html>
003            <head>
004                    <title>DOM</title>
005                    <script>
006                            var textareaSalida;
007                            function verDOM(nodo, sangrado) {
008                                    var i;
009                                    textareaSalida.value += sangrado + nodo.nodeType +
       '-' + nodo.nodeName + '\n';
010                                    for ( i = 0; i < nodo.childNodes.length; i++) {
011                                            if (nodo.childNodes[i].childNodes.length >
       0) {
012                                                    verDOM(nodo.childNodes[i], sangrado +
       ' ');
013                                            } else {
014                                                    textareaSalida.value += sangrado + '
       ' + nodo.childNodes[i].nodeType + '-' + nodo.childNodes[i].nodeName + '\n';
015                                            }
016                                    }
017                            }
018                    </script>
019            </head>
020            <body>
021                    <p>
022                            Pulse el bot&oacute;n para descubir su <strong>DOM</strong>
023                    </p>
024                    <br />
025                    <button type='button' onclick='var sangrado ="";textareaSalida =
       document.getElementById("salida");textareaSalida.value="";verDOM(document,sangrad
       o);'>
026                            Ver DOM
027                    </button>
028                    <br />
029                    <textarea id='salida'></textarea>
030            </body>
031    </html>
```

V2 eliminando intros y espacios

```
<!DOCTYPE html><html><head><title>DOM</title><script>
var textareaSalida;
function verDOM(nodo, sangrado) {
var i;
textareaSalida.value += sangrado + nodo.nodeType + '-' + nodo.nodeName + '\n';
for ( i = 0; i < nodo.childNodes.length; i++) {
if (nodo.childNodes[i].childNodes.length > 0) {
verDOM(nodo.childNodes[i], sangrado + ' ');
} else {
textareaSalida.value += sangrado + ' ' + nodo.childNodes[i].nodeType + '-' + nodo.childNodes[i].nodeName + '\n';
}
}
}
</script></head><body><p>Pulse el bot&oacute;n para descubir su <strong>DOM</strong></p><br/><button type='button' onclick='var sangrado ="";textareaSalida = document.getElementById("salida");textareaSalida.value="";verDOM(document,sangrado);'>Ver DOM</button><br/><textarea id='salida'></textarea></body></html>
```

Al probar este ejemplo obtendríamos un árbol DOM como el que se muestra a continuación.

```
9-#document
 10-html
 1-HTML
  1-HEAD
   3-#text
   1-TITLE
    3-#text
   3-#text
   1-SCRIPT
    3-#text
   3-#text
  3-#text
  1-BODY
   3-#text
   1-P
    3-#text
    1-STRONG
     3-#text
    3-#text
   3-#text
   1-BR
   3-#text
   1-BUTTON
    3-#text
   3-#text
   1-BR
   3-#text
   1-TEXTAREA
   3-#text
```



Nodos V2

Lo único que puede sorprendernos de este resultado es **la abundancia de nodos de tipo texto**; estos nodos **se deben a los saltos de línea** del código.

En cierta forma JavaScript es el titiritero que controla la página web a través de los hilos que le tiende el DOM.

Nota: Otro método muy utilizado del objeto `document` es `write(cadena)`, que inserta en el documento la cadena de caracteres especificada.
